



Tecnológico de Monterrey

Software Requirements Specification

Software Construction and Decision Making

Grupo 501

Daniela Janet Gil Gonzalez
Yuhao Liu
Mariana Ramirez Cervera

A01752908
A01787782
A01787819

6 de mayo del 2026

Software Requirements Specification

a. Functional requirements

1. The map should change every time I start a new game
2. There must be at least 3 different levels through the game
3. There should be different character options with unique functionality.
4. There must be a timer bar system for each combat
5. There must be a reward system after each combat
6. I would like to save the progress of cards won
7. There should be math mechanics to fight each enemy
8. Math problems must be solved for each combat
9. There must be random chests along the game
10. There should be attack cards to damage enemies
11. There should be defense cards to reduce incoming damage
12. There must be special skill cards with unique abilities
13. Health bars should display the player and enemy HP
14. There should be a functional main menu
15. Enemies should have unique abilities and behaviors
16. There must be boss battles at the end of each world
17. The type of math operations should vary depending on the map level
18. Players should be able to choose their deck cards before each combat
19. Cards must change and be substituted after activating them successfully in a combat
20. There must be feedback for correct answers
21. There must be feedback for incorrect answers
22. There should be a progression system throughout the game
23. Players should retain part of their progress after losing a run
24. Players should be able to choose the game difficulty.
25. Players should be able to select preferred math operations in order to practice a specific type.

b. Non-functional

1. The game interface should be clear and easy to understand for children between 8 and 14 years old.
2. The text, buttons, math problems, health bars, and timer should be easy to read during the game.
3. The game should use clear colors and visual elements so the player can understand what is happening.
4. The timer bar should clearly show the player's performance using colors such as green, yellow, and red.

5. The game should give clear feedback when the player answers correctly or incorrectly.
6. The game should run smoothly in a modern web browser without freezing or slowing down during gameplay.
7. The game should save the player's progress correctly, including cards, skill cards, and unlocked progress.
8. The saved progress should not be lost when the player refreshes the page or returns to the main menu.
9. The game should have a consistent pixel-art visual style in characters, enemies, cards, maps, and backgrounds.
10. The character sprites should be visually attractive and different enough for the player to recognize them.
11. The sound effects should help the player understand important actions, such as card activation, correct answers, wrong answers, attacks, victory, and defeat.
12. The player should be able to adjust or control the background music and sound effects.
13. The game should be compatible with common modern browsers such as Chrome, Edge, Firefox, or Safari.
14. The controls should be simple and intuitive, using the mouse for menus and card selection, and the keyboard for math answers.
15. The difficulty should increase gradually so the game stays challenging but does not become too stressful for the target audience.
16. The code should be organized in a clear way so the team can maintain it and add new content more easily.
17. The repository should include clear installation instructions so instructors can run and test the game without confusion.
18. The game should not store sensitive personal information from the players.

c. User Stories:

User story #1 Random Map Generation As a user i want the map to change every time I start a new game so that I can have a different experience in every run	
Validation: • Verify that enemies change between runs.	Priority: high Estimation: 8 hours

• Design a graph map where every node is an event • Check that players can choose different paths.	
---	--

User story #2 Multiple levels

As a user

i want at least 3 different levels in the game

so that I can experience progression and new challenges.

Validation:

- Verify the existence of at least 3 different maps.
- The player can successfully change between levels
- The levels have no order restrictions for choosing them

Priority: High

Estimation: 10 hrs

User story #3 Character Selection

As a user

i want to choose between different characters

so that I can personalize my gameplay experience with their functions.

Validation:

- Characters could: have more seconds on the timer, display easier math problems, or have more effect on conditions
- Confirm the functionality of the 3 of them
- Check balance and adjust values

Priority: Medium

Estimation: 8 hrs

User story #4 Timer Bar System

As a user

i want a timer system during math challenges

so that the game feels more challenging and engaging.

Validation:

- Verify timer functionality.
- Confirm timer color changes correctly.
- Check reduced effectiveness when answers are slower.

Priority: High

Estimation: 6 hrs

User story #5 Reward System

As a user

i want new cards after winning battles

so that I feel progression and achievement.

Validation:

- Verify rewards are granted correctly.
- Confirm new cards unlock.
- Check level progression functionality.

Priority: High

Estimation: 8 hrs

User story #6 Save Card Progress

As a user

i want to save my reward cards

so that I can keep track of my improvement and save game progression

Validation:

- Verify card progress is stored correctly.
- Confirm saved data loads properly.

Priority: High

Estimation: 10 hrs

• Check persistence of cards.	
---	--

User story #7 Combat System

As a user

i want to fight enemies using cards and math mechanics
so that I can practice progress through the game.

Validation:

- Verify card activation works properly.
- Confirm enemies receive damage correctly.
- Check combat turn system functionality.

Priority: High

Estimation: 14 hrs

User story #8 Math problem system

As a user

i want to solve math problems during combat
so that I can activate defense and attack cards to fight enemies.

Validation:

- Verify math problems are generated correctly.
- Confirm answer validation works.
- Check progressive difficulty scaling.

Priority: High

Estimation: 10 hrs

User story #9 Random chests

As a user

i want to encounter random chests through a map
so that each run feels unique and unpredictable.

Validation: • Verify chest node spawning in the graph. • Confirm random events trigger correctly. • Check reward and trap functionality. • The player can choose a chest node.	Priority: High Estimation: 10 hrs
--	--

User story #10 Attack Cards As a user i want to use attack cards against enemies so that I can defeat enemies faster.	
Validation: • Verify attack cards deal damage depending on the timer. • Confirm attack animations trigger correctly. • Check enemy HP reduction.	Priority: High Estimation: 10 hrs

User story #11 Defense Cards As a user i want to use defense cards with different functions so that I can reduce incoming damage and protect myself.	
Validation: • Verify that defense effects work depending on the timer. • Design functions such as damage reflection for the enemy or healing for the player • Confirm unique function applies	Priority: High Estimation: 10 hrs

correctly..	
-------------	--

User story #12 Skill Cards

As a user

i want to unlock special skill cards

so that I can create stronger strategies.

Validation:

- Verify skill cards unlock properly only after winning against a final boss.
- Confirm special functions work correctly.
- Check deck restrictions for skill cards (only 1 per combat).

Priority: High

Estimation: 10 hrs

User story #13 Health Bars

As a user

i want to see health bars during combat

so that I can track battle progress.

Validation:

- Verify HP bars update correctly.
- Confirm health bars are visible.
- Check combat synchronization.

Priority: High

Estimation: 6 hrs

User story #14 Main menu

As a user

i want a functional main menu

so that I can navigate the game easily.

Validation: • Verify all buttons function properly (play, instructions, load game, stats, options) .	Priority: High Estimation: 2 hrs
---	---

User story #15 Enemies with varied characteristics As a user i want every enemy to have unique abilities so that combat feels more varied and interesting.	
Validation: • Verify unique enemy abilities. • Confirm special functions activate correctly. • Check gameplay balance.	Priority: High Estimation: 14 hrs

User story #16 Boss Battles As a user i want boss fights at the end of each world so that I can experience more challenges.	
Validation: • Verify boss encounters trigger correctly (at the final combat of each level). • Confirm unique boss mechanics. • Check rewards after defeating bosses (skill cards).	Priority: High Estimation: 14 hrs

User story #17 Different Map Difficulty As a user	
--	--

i want the game difficulty to increase gradually
so that I continue feeling challenged as I progress.

Validation:

- Verify card activation difficulty scaling.
- 1st level must have addition and subtraction, 2nd multiplication and division, and 3rd mixed operations

Priority: High

Estimation: 8 hrs

User story #18 Combat Deck of cards

As a user

i want to choose my initial deck before starting a combat with an enemy
so that i can change my strategy and defeat the enemy.

Validation:

- Verify available cards for the combat.
- Confirm the selection of cards.

Priority: High

Estimation: 8 hrs

User story #19 Changing cards

As a user

i want the initial deck to switch with my other cards once I activate them
so that the combat feels challenging and varied.

Validation:

- Verify that cards only switch after activating correctly
- Cards switched are only from the player's deck
- Check animation smoothness.

Priority: High

Estimation: 4 hrs

User story #20 Correct Answer Feedback

As a user

i want feedback when I answer correctly
so that I know my action in the combat succeeded.

Validation:

- Verify positive feedback text.
- Make sure the text is readable
- Combine the ‘Correct!’ text with the effect applied.

Priority: low

Estimation: 0.5 hrs

User story #21 Incorrect Answer Feedback

As a user

i want feedback when I answer incorrectly
so that I understand if I made a mistake.

Validation:

- Verify incorrect answer texts.
- Display text for time out and for wrong answer
- Show null effect text..

Priority: Low

Estimation: 1 hrs

User story #22 Progression System

As a user

i want to improve as I progress through the game
so that I feel rewarded for continuing to play.

Validation:

- Verify leveling system.
- Check unlockable content.
- See statistics of wins and losses.

Priority: High

Estimation: 9 hrs

User story #23 Retain Progress After Losing

As a user

i want to keep part of my progress after losing a run
so that I do not feel like all my effort was lost.

Validation:

- Verify that the player loses normal deck cards after losing.
- Verify that the player keeps unlocked Skill cards.
- Confirm that saved progress loads correctly after starting again.

Priority: High

Estimation: 8 hrs

User story #24 Difficulty Selection

As a user

i want to choose the game difficulty
so that I can play at a level that matches my math skills.

Validation:

- Verify that the difficulty option appears in the options menu.
- Confirm that the selected difficulty changes the math problem level.
- Check that the selected difficulty is applied during combat..

Priority: Medium

Estimation: 6 hrs

User story #25 Preferred Math Operations

As a user

i want to select the type of math operations I want to practice
so that I can improve specific skills, such as addition, subtraction, multiplication, or division.

Validation: • Verify that the player can choose preferred math operations. • Confirm that the game generates problems based on the selected operation. • Check that the selected operation is used during combat.	Priority: medium Estimation: 6 hrs
--	---

User story #26 Character Sprites As a user i want to choose between different character types or skins so that I can personalize my gameplay experience.	
Validation: • Verify multiple selectable characters exist. • Confirm selected skins appear during gameplay. • Check functionality of the character selection screen.	Priority: Medium Estimation: 4 hrs

d. Organization:

The project was organized using GitHub as the platform for development. The repository includes all the files needed to run, review, and test the game. The files were divided into folders according to their purpose, which makes the project easier to understand and maintain.

The main code of the game was placed in the src folder. Inside this folder, the team organized different parts of the game, such as scenes, systems, entities, cards, assets, and user interface elements. This structure helps separate the different responsibilities of the code and makes it easier to find or modify specific parts of the project.

The repository also includes folders for the backend, database, and testing files. The backend folder contains the files needed for the server side of the project. The database folder includes

the files related to the database structure and initial data. The testing folder was included to support the review and verification of the project.

The README file explains the purpose of the project and the steps needed to install and run the game. This is important because it allows instructors or other users to test the game without needing extra explanations from the team. The README also helps explain the organization of the repository.

GitHub Issues were used to divide the project into smaller tasks. Each issue represented a specific feature, correction, or improvement related to the game. The issues were based on the user stories and functional requirements, so the work in GitHub was connected to the requirements document.

The GitHub Project board was used to organize the development process. The board included columns such as Product Backlog, Sprint Backlog, In Progress, Testing, and Done. These columns helped the team track the status of each task during the project.

The Product Backlog included tasks that had not been started yet. The Sprint Backlog included the tasks selected for a specific sprint. The In Progress column showed the tasks being developed. The Testing column included tasks that needed to be reviewed before completion. Finally, the Done column included the tasks that were already finished.

This organization helped the team follow a clear workflow. It also made the project easier to manage because each task had a specific place in the board. In this way, the repository, issues, and project board worked together to show the planning, development, and completion of the game.